Python Notes: If Statements in Python 🤔

`if`, `elif`, and `else` — Decision Making

Video: If statements in Python are easy (if, elif, else) 🤔
Video URL: https://www.youtube.com/watch?v=FvMPfrgGeKs
Main topics: Conditional statements, comparison operators, indentation, `Boolean values`, and decision-making programs.

---

1. What Are If Statements?

An **if statement** is a conditional statement used to execute a block of code only when a specified condition is `True`.

Python uses conditional statements to make decisions based on different situations.

For example:

- Check whether a person is old enough to sign up for a service.
- Determine whether a number is positive or negative.
- Check whether a user entered their name.
- Decide whether an item is available for sale.
- Check whether a user is online or offline.

Real-life example

Imagine a rule:

> *If a student scores at least `40` marks, they pass. Otherwise, they fail.*

In Python:

2. The Three Main Conditional Statements

Python provides three commonly used keywords for conditional decision-making.

Keyword	Purpose
`if`	Checks the first condition.
`elif`	Checks another condition if the preceding conditions are false.
`else`	Executes when none of the preceding conditions are true.

> 💡 MEMORY TRICK
>
> Remember: `elif` is short for **else if**.

2.1 The `if` Statement

An `if` statement executes its indented block only when its condition is true.

Syntax

```
if condition:
    # Code executes if condition is True
```

Example

```
age = 21

if age >= 18:
    print("You are eligible to sign up.")
```

Output:

```
You are eligible to sign up.
```

2.2 The `else` Statement

An `else` statement provides an alternative block of code when the preceding `if` condition is false.

Syntax

```
if condition:
    # Runs if condition is True
else:
    # Runs if condition is False
```

Example: Age eligibility

```
age = int(input("Enter your age: "))

if age >= 18:
    print("You are eligible to sign up.")
else:
    print("You must be at least 18 to sign up.")
```

Sample execution 1:

```
Enter your age: 21
You are eligible to sign up.
```

Sample execution 2:

```
Enter your age: 13
You must be at least 18 to sign up.
```

How does it work?

1. `input()` accepts the user's age as a string.
2. `int()` converts the input into an integer.

2.3 The `elif` Statement

The `elif` statement allows you to check additional conditions between an `if` and an optional `else`.

Use `elif` when the program needs to choose between multiple possibilities.

Syntax

```
if condition1:
    # Runs if condition1 is True
elif condition2:
    # Runs if condition1 is False
    # and condition2 is True
elif condition3:
    # Runs if the previous conditions are False
    # and condition3 is True
else:
    # Runs if every preceding condition is False
```

You can use multiple `elif` statements in the same conditional structure.

Example: Age categories

```
age = int(input("Enter your age: "))

if age < 0:
    print("Invalid age.")
elif age < 18:
    print("You are a minor.")
elif age < 60:
    print("You are an adult.")
else:
    print("You are a senior adult.")
```

3. The Importance of Condition Order

The order of conditions is extremely important when using `if` and `elif`.

Python does not automatically rearrange conditions according to their importance. It checks them in the order you write them.

Example: A sign-up age check

Suppose a service accepts users aged 18 or older but does not allow anyone aged 100 or above.

Incorrect program:

```
age = 111

if age >= 18:
    print("You are eligible to sign up.")
elif age >= 100:
    print("You are too old to sign up.")
else:
    print("You must be at least 18 to sign up.")
```

Output:

```
You are eligible to sign up.
```

Why? Because `111 >= 18` is already true. Python executes that block and skips the `elif` condition.

4. Comparison Operators

Comparison operators compare two values and return a Boolean value: `True` or `False`.

They are frequently used inside conditional statements.

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal to	`5 != 3`	`True`
`>`	Greater than	`8 > 4`	`True`
`<`	Less than	`3 < 7`	`True`
`>=`	Greater than or equal to	`5 >= 5`	`True`
`<=`	Less than or equal to	`4 <= 2`	`False`

4.1 Difference Between `=` and `==`

This is one of the most important concepts in Python.

Assignment operator: `=`

The single equals sign assigns a value to a variable.

```
response = "y"
```

Here, `"y"` is assigned to **`response`**.

Comparison operator: `==`

The double equals sign compares two values to check whether they are equal.

5. Example: Asking the User About Food 🍕

The video demonstrates a simple program that asks whether the user wants food.

The user enters `y` for yes or `n` for no.

Complete program

```
response = input("Would you like some food? (y/n): ")

if response == "y":
    print("Have some food!")
else:
    print("No food for you.")
```

Sample execution 1

```
Would you like some food? (y/n): y
Have some food!
```

Sample execution 2

```
Would you like some food? (y/n): n
No food for you.
```

Explanation

- `input()` accepts the user's response.
- `response == "y"` checks whether the response is exactly the lowercase string `"y"`.
- If it matches, the `if` block executes.
- Otherwise, the `else` block executes.

Important: This beginner example treats every response other than lowercase `"y"` as a no. That includes invalid responses, such as `"maybe"`.

6. Example: Checking Whether a Name Is Empty

The video also demonstrates how to check whether a user has entered a name.

An **empty string** is a string containing no characters:

```
""
```

Its length is zero.

Program

```
name = input("Enter your name: ")

if name == "":
    print("You did not enter your name.")
else:
    print(f"Hello, {name}!")
```

Sample execution 1: Empty input

```
Enter your name:
You did not enter your name.
```

Sample execution 2: A name is entered

```
Enter your name: Ashwani
Hello, Ashwani!
```

Explanation

- `name == ""` checks whether the name is an empty string.
- If it is empty, Python displays a warning.
- Otherwise, the program greets the user using an f-string.

7. Using Boolean Variables in If Statements

A **Boolean** is a data type that has only two possible values:

- `True`
- `False`

If a variable already contains a Boolean value, you can use it directly in an `if` statement without comparing it to `True` or `False`.

7.1 Example: An item for sale

```
for_sale = True

if for_sale:
    print("This item is for sale.")
else:
    print("This item is not for sale.")
```

Output:

```
This item is for sale.
```

Because `for_sale` is `True`, the first block executes.

If the variable is changed to `False`:

```
for_sale = False

if for_sale:
    print("This item is for sale.")
else:
    print("This item is not for sale.")
```

Output:

8. Indentation in Python

Indentation means adding spaces at the beginning of a line of code.

Python uses indentation to identify which statements belong to a conditional block.

Correct indentation

```
age = 20

if age >= 18:
    print("You are an adult.")
    print("You can continue.")
```

Both `print()` statements are indented, so both belong to the `if` block.

Incorrect indentation

```
age = 20

if age >= 18:
print("You are an adult.")
```

This causes an `IndentationError`.

Important indentation rules

1. Use a colon (`:`) after the condition.
2. Indent the statements that belong to the block.
3. Use consistent indentation. Four spaces per indentation level is the standard convention.
4. Keep `elif` and `else` aligned with their corresponding `if`.
5. Do not indent a statement unless it belongs to an enclosing block.

9. Putting It All Together: Conditional Flow

Consider this program:

```
score = 75

if score >= 90:
    print("Grade A")
elif score >= 75:
    print("Grade B")
elif score >= 60:
    print("Grade C")
else:
    print("Needs improvement")
```

Output:

```
Grade B
```

Python checks the branches from top to bottom:

1. `score >= 90` → false.
2. `score >= 75` → true.
3. Python prints `"Grade B"` and skips the remaining branches.

Notice that `score >= 60` is also true, but its block does not execute because an earlier branch already matched.

This is a central feature of an **`if-elif-else`** chain.

10. Common Mistakes to Avoid

Mistake 1: Using `=` instead of `==`

Incorrect:

Mistake 4: Placing conditions in the wrong order

Incorrect:

```
if age >= 18:
    print("Adult")
elif age >= 100:
    print("Age limit exceeded")
```

Someone aged 100 or older will match the first condition. If the second condition must take priority, check it first.

Mistake 5: Assuming `else` needs a condition

Incorrect:

```
else age < 18:
    print("Minor")
```

Correct:

```
else:
    print("Minor")
```

The `else` branch runs when all previous conditions in the chain are false, so it does not require its own condition.

Mistake 6: Forgetting that `input()` returns a string

Incorrect:

```
age = input("Enter your age: ")

if age >= 18:
    print("Eligible")
```

This raises a `TypeError` because `age` is a string and `18` is an integer.

Question 3

```
number = 0

if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

Question 4

```
response = "y"

if response == "yes":
    print("Accepted")
else:
    print("Rejected")
```

Question 5

```
score = 95

if score >= 60:
    print("Passed")
elif score >= 90:
    print("Excellent")
else:
    print("Failed")
```

Why does this code print `"Passed"` rather than `"Excellent"`?

Answer key

Question	Correct answer
1	`A`
2	`Minor`
3	`Zero`
4	`Rejected`
5	`Passed`, because the first condition is true and Python skips the later branches.
6	`Offline`

---

12. Mini Project: Simple Eligibility Checker

Build a program that asks the user for their age and categorizes the result.

Requirements

- Ask the user to enter their age.
- Display an invalid-age message for negative ages.
- Display a children's category for ages below 13.
- Display a teenager category for ages from 13 to 17.
- Display an adult category for ages 18 and above.

13. Last-Minute Revision Sheet

Essential syntax

`if` statement

```
if condition:
    statement
```

`if-else` statement

```
if condition:
    statement1
else:
    statement2
```

`if-elif-else` statement

```
if condition1:
    statement1
elif condition2:
    statement2
else:
    statement3
```

Must-remember points

1. `if` executes a block when its condition is true.
2. `elif` checks another condition if the preceding conditions were false.
3. `else` provides a default branch when no preceding condition is true.
4. Conditions are evaluated from top to bottom.
5. Only the first matching branch executes in a single `if-elif-else` chain.
6. Use `==` to compare values and `=` to assign values.
7. Python uses indentation to define code blocks.

8. A Boolean variable can be used directly: `if online:`.
9. `input()` returns a string, so use `int()` or `float()` when numeric comparisons or calculations are needed.
10. Arrange overlapping conditions carefully to ensure the intended branch executes.

Final takeaway

Conditional statements are one of the fundamental building blocks of Python programming. Once you understand `if`, `elif`, `else`, comparison operators, Boolean values, and indentation, you can write programs that make decisions instead of simply executing every statement in sequence.

Practise the age checker, food-response program, and name-validation example until you can write each one without referring to these notes.

